Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №15
Разработка веб-приложения на Python с использованием Flask

1. Цель работы

Получить практические навыки разработки веб-приложений на Python с использованием микрофреймворка Flask: изучить архитектуру Flask-приложения, создавать маршруты и обработчики HTTP-запросов, работать с параметрами URL и query-параметрами, формировать HTML-страницы с помощью шаблонов Jinja2 и JSON-ответы, реализовать CRUD-операции, подключить реляционную базу данных через SQLAlchemy, разделить веб-слой, бизнес-логику и доступ к данным, а также объединить HTML-интерфейс и REST API в одном приложении с серверным рендерингом.

2. Краткие теоретические сведения

Flask — минималистичный веб-фреймворк для Python. В отличие от Django он предоставляет только базовые механизмы веб-приложения, а дополнительные компоненты подключаются отдельно. Минимальное приложение состоит из экземпляра Flask и маршрута: @app.route("/") связывает URL с функцией-обработчиком. Маршруты могут содержать параметры (<int:dish_id>) и поддерживать несколько HTTP-методов (methods=["GET", "POST"]).

Из запроса можно получать query-параметры (request.args.get), данные форм (request.form.get) и JSON (request.get_json); ответ формируется как строка, HTML или jsonify с HTTP-кодом. Jinja2 обеспечивает серверный рендеринг HTML: шаблоны наследуются (base.html с блоками title и content), коллекции перебираются циклом {% for %}, условия — {% if %}, а значения подставляются через {{ }}.

Архитектура многослойного Flask-приложения: клиент отправляет HTTP-запрос, Flask направляет его в маршрут, который вызывает view-функцию; далее следуют слои Service (бизнес-логика), Repository (доступ к данным) и SQLAlchemy ORM (база данных). Для разделения маршрутов используются

Blueprint (web и api), а для разных конфигураций и тестирования — паттерн Application Factory: функция create_app() создаёт и настраивает приложение.

3. Индивидуальное задание

Вариант 5 — предметная область «Ресторан». Сущности: Dish (блюдо), Category (категория), Order (заказ). Требуется реализовать веб-приложение с HTML-интерфейсом (главная страница, список блюд, карточка блюда, формы создания и редактирования, страница заказов) и REST API; полный CRUD; поиск по названию/описанию; фильтрацию по категории и доступности; сортировку по названию, цене и калорийности в двух направлениях; работу со связанными сущностями (блюда в заказе, категория блюда); хранение данных в SQLAlchemy; обработку ошибок (404, 400, 409); конфигурацию через переменные окружения.

4. Архитектура приложения

Приложение построено по многослойной схеме с двумя точками входа: HTML-интерфейс (web.py) и REST API (api.py) обращаются к общим слоям Service → Repository → SQLAlchemy. Схема приведена на рисунке 15.1.

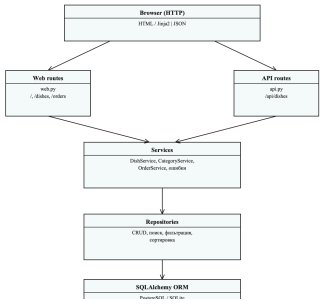


Рисунок 15.1 – Архитектура Flask-приложения

```
lab_15/
├── app/
│   ├── __init__.py            # Application Factory create_app, seed данных
│   ├── config.py              # настройки через переменные окружения
│   ├── routes/
│   │   ├── web.py             # HTML-интерфейс (Jinja2)
```

```
│   │   └── api.py              # REST API (JSON)
│   ├── models/models.py        # Category, Dish, Order (связь N:M через
order_items)
│   ├── repositories/repositories.py  # доступ к данным
│   ├── services/services.py    # бизнес-логика и ошибки
│   ├── templates/              # base, index, list, detail, create, edit,
orders, 404
│   └── static/css/style.css
├── tests/test_app.py           # 26 тестов
├── requirements.txt
└── README.md
```

Приложение собирается функцией create_app() (Application Factory): создаётся экземпляр Flask, загружается конфигурация, инициализируется SQLAlchemy, регистрируются Blueprint web и api, настраивается логирование, регистрируется обработчик 404 и при первом запуске создаются таблицы и тестовые данные. Такой подход удобен для тестирования и позволяет создавать разные конфигурации приложения.

5. ORM-модели и база данных

Модель данных ресторана: Category (категория, уникальное название), Dish (блюдо: название, описание, цена, калорийность, доступность, внешний ключ category_id), Order (заказ: имя клиента, статус, дата) с связью N:M к блюдам через таблицу order_items. Такая схема соответствует предметной области и демонстрирует работу со связанными сущностями: у блюда есть категория, заказ содержит несколько блюд.

Файл: app/models/models.py

```
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()


class Category(db.Model):
    """Категория блюда."""

    __tablename__ = "categories"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(80), unique=True, nullable=False)

    dishes = db.relationship("Dish", back_populates="category", cascade="all,
delete-orphan",
                             lazy="selectin")

    def __repr__(self):
        return f"Category({self.name!r})"


class Dish(db.Model):
    """Блюдо меню."""

    __tablename__ = "dishes"
```

```
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(120), nullable=False)
    description = db.Column(db.Text, default="")
    price = db.Column(db.Float, nullable=False)
    calories = db.Column(db.Integer, nullable=False, default=0)
    available = db.Column(db.Boolean, nullable=False, default=True)

    category_id = db.Column(db.Integer, db.ForeignKey("categories.id"),
nullable=False)
    category = db.relationship("Category", back_populates="dishes")

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "name": self.name,
            "description": self.description,
            "price": self.price,
            "calories": self.calories,
            "available": self.available,
            "category_id": self.category_id,
            "category_name": self.category.name if self.category else None,
        }

    def __repr__(self):
        return f"Dish({self.name!r}, {self.price})"


order_items = db.Table(
    "order_items",
    db.Column("order_id", db.Integer, db.ForeignKey("orders.id"),
primary_key=True),
    db.Column("dish_id", db.Integer, db.ForeignKey("dishes.id"),
primary_key=True),
    db.Column("quantity", db.Integer, nullable=False, default=1),
)


class Order(db.Model):
```

Подключение к базе: Flask-SQLAlchemy (db.init_app(app)); строка подключения задаётся через переменную окружения DATABASE_URL (по умолчанию SQLite restaurant.db, для PostgreSQL — postgresql+psycopg://...). Таблицы создаются автоматически при старте приложения.

6. Доступ к данным (Repository) и бизнес-логика (Service)

Репозитории изолируют SQLAlchemy-запросы: поиск по названию и описанию (contains), фильтрация по категории и доступности, сортировка по названию/цене/калорийности с направлением asc/desc, CRUD блюд, категорий и заказов. Сервисы содержат бизнес-правила: непустое название, положительная цена, существование категории и блюда, допустимые статусы заказа, корректный набор блюд в заказе. Ошибки представлены пользовательскими исключениями NotFoundError, ConflictError, ValidationError.

Файл: app/repositories/repositories.py (фрагмент)

```
class DishRepository:
```

```
    """Доступ к блюдам: CRUD, поиск, фильтрация, сортировка."""

    def get_all(self) -> List[Dish]:
        return list(Dish.query.order_by(Dish.name).all())

    def get(self, dish_id: int) -> Optional[Dish]:
        return db.session.get(Dish, dish_id)

    def search(self, search: Optional[str], category_id: Optional[int],
               available: Optional[bool], sort: str, order: str) -> List[Dish]:
        query = Dish.query
        if search:
            query = query.filter(or_(Dish.name.contains(search),
Dish.description.contains(search)))
        if category_id:
            query = query.filter(Dish.category_id == category_id)
        if available is not None:
            query = query.filter(Dish.available == available)

        column = {"name": Dish.name, "price": Dish.price, "calories":
Dish.calories}.get(sort, Dish.name)
        if order == "desc":
            column = column.desc()
        return list(query.order_by(column).all())

    def create(self, name: str, description: str, price: float, calories: int,
               available: bool, category_id: int) -> Dish:
```

Файл: app/services/services.py (фрагмент)

```
        self._categories = category_repository

    def list(self, search: Optional[str] = None, category_id: Optional[int] =
None,
             available: Optional[bool] = None, sort: str = "name",
             order: str = "asc") -> List[Dish]:
        return self._repository.search(search, category_id, available, sort,
order)

    def get(self, dish_id: int) -> Dish:
        dish = self._repository.get(dish_id)
        if dish is None:
            raise NotFoundError(f"Блюдо с ID {dish_id} не найдено")
        return dish

    def create(self, name: str, description: str, price: float, calories: int,
               available: bool, category_id: int) -> Dish:
        if not name.strip():
            raise ValidationError("Название блюда не может быть пустым")
        if price <= 0:
            raise ValidationError("Цена должна быть положительной")
        if calories < 0:
            raise ValidationError("Калорийность не может быть отрицательной")
        if self._categories.get(category_id) is None:
            raise ValidationError(f"Категория с ID {category_id} не существует")
        return self._repository.create(name, description, price, calories,
available, category_id)

    def update(self, dish_id: int, fields: dict) -> Dish:
        self.get(dish_id)
```

7. HTML-интерфейс (Jinja2)

Интерфейс построен на базовом шаблоне base.html: навигационное меню (главная, меню, добавить блюдо, заказы), блок сообщений flash и блок content, который переопределяется на каждой странице. Страницы: главная со статистикой заказов; список блюд с формой поиска, фильтрации и сортировки и

таблицей; карточка блюда; формы создания и редактирования; таблица заказов со сменой статуса; собственный шаблон 404. Оформление — в static/css/style.css.

Страница	URL	Назначение
Главная	/	статистика заказов
Список блюд	/dishes	поиск, фильтр, сортировка
Карточка блюда	/dishes/<id>	подробная информация
Создание	/dishes/create	форма добавления блюда
Редактирование	/dishes/<id>/edit	форма изменения
Заказы	/orders	список заказов, статусы
Новый заказ	/orders/create	оформление заказа
404	—	обработчик отсутствующей страницы

Файл: app/templates/base.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{% block title %}Ресторан{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/
style.css') }}">
</head>
<body>
<header>
    <nav class="navbar">
        <a class="brand" href="{{ url_for('web.index') }}">🍽 Ресторан</a>
        <a href="{{ url_for('web.index') }}">Главная</a>
        <a href="{{ url_for('web.list_dishes') }}">Меню</a>
        <a href="{{ url_for('web.create_dish') }}">Добавить блюдо</a>
        <a href="{{ url_for('web.list_orders') }}">Заказы</a>
        <a href="{{ url_for('web.create_order') }}">Новый заказ</a>
    </nav>
</header>
<main class="container">
    {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
            {% for category, message in messages %}
                <div class="flash flash-{{ category }}">{{ message }}</div>
            {% endfor %}
        {% endif %}
    {% endwith %}
    {% block content %}{% endblock %}
</main>
<footer>Лабораторная работа №15 — Flask-приложение «Ресторан»</footer>
```

Примеры работы HTML-интерфейса приведены на рисунке 15.2.

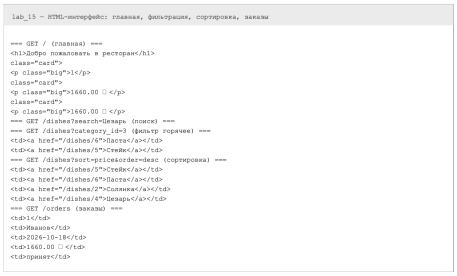


Рисунок 15.2 – HTML-интерфейс: главная, фильтрация, сортировка, заказы

8. REST API

Помимо HTML-интерфейса реализован REST API в отдельном Blueprint api (префикс /api). API возвращает JSON с корректными HTTP-кодами: 200, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, 409 Conflict. Формат ошибок единый: {"error": "текст"}.

Метод	URL	Назначение	Код
GET	/api/dishes	список блюд (поиск, фильтры, сортировка)	200
GET	/api/dishes/<id>	блюдо по ID	200/404
POST	/api/dishes	создание блюда	201/400
PUT	/api/dishes/<id>	изменение блюда	200/404
DELETE	/api/dishes/<id>	удаление блюда	204/404
GET	/api/categories	список категорий	200
GET	/api/orders	список заказов	200
POST	/api/orders	создание заказа	201/400

Демонстрация REST API (CRUD, создание заказа, ошибка 404) приведена на рисунке 15.3.

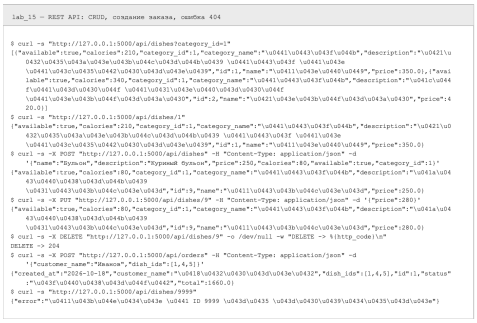


Рисунок 15.3 – REST API: CRUD, создание заказа, ошибка 404

9. Обработка ошибок и конфигурация

Приложение обрабатывает типичные ошибки: 404 — собственным HTML-шаблоном (web) и JSON-ответом (api); 400 и 409 — через пользовательские исключения ValidationError и ConflictError, преобразованные в HTTP-коды; ошибки валидации форм выводятся пользователю как flash-сообщения. Настройки (DATABASE_URL, SECRET_KEY) задаются переменными окружения и описаны в .env.example; реальные секреты не хранятся в коде. Ведётся журнал: запуск приложения, создание и изменение объектов, ошибки — через стандартный logging.

10. Демонстрация работы

Запуск приложения: ../venv/bin/python -m app — сервер на http://127.0.0.1:5000. При первом запуске база заполняется тестовыми данными: 5 категорий, 8 блюд; сайт доступен в браузере, API — по адресам /api/*. Полный сценарий: главная страница → список блюд → поиск и фильтрация → карточка блюда → создание и редактирование → заказ → изменение статуса → операции через REST API.

Взаимодействие с приложением продемонстрировано на рисунках 15.2–15.3; изменение данных сохраняется в базе SQLAlchemy и отображается как в HTML-интерфейсе, так и в API.

11. Тестирование

Написано 26 автоматических тестов pytest с использованием test_client Flask: страницы (главная, список, детальная, 404), поиск, фильтрация по категории, сортировка, создание/редактирование/удаление блюда через формы, валидация (неположительная цена, заказ без блюд), изменение статуса заказа, REST API (список, получение, создание, PUT, DELETE, ошибки 400/404, категории, создание заказа с расчётом суммы и ошибка для несуществующего блюда). Результат выполнения pytest приведён на рисунке 15.4.

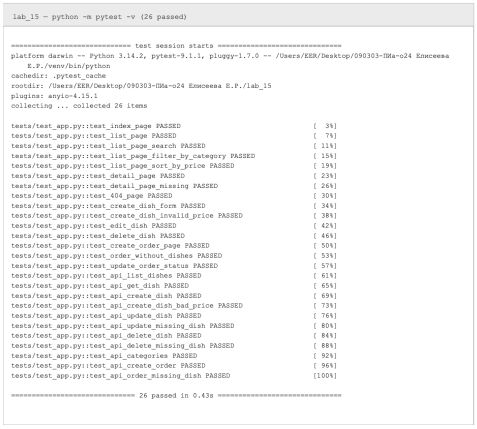


Рисунок 15.4 – Результаты выполнения pytest (26 passed)

12. Вывод

В ходе лабораторной работы разработано полноценное веб-приложение «Ресторан» на Flask, объединяющее HTML-интерфейс с серверным рендерингом Jinja2 и REST API: реализованы маршруты и обработчики, CRUD, поиск, фильтрация и сортировка, работа со связанными сущностями (категории, заказы с блюдами), хранение данных через SQLAlchemy, обработка ошибок, конфигурация через переменные окружения и логирование.

Приложение построено по многослойной схеме Blueprint → Service → Repository → SQLAlchemy с паттерном Application Factory, а 26 автоматических тестов покрывают HTML-страницы, формы, валидацию, REST API и обработку ошибок. Выполненная работа показала разницу между минималистичным Flask и изученными ранее FastAPI и Django и дала практические навыки построения веб-приложения, где веб-слой отделён от бизнес-логики и доступа к данным.